Lesson 17 — Camera Geometry, Pixels and Depth

Module 4: Robot Perception | Status: Completed

Learning outcomes

- Explain why a 2D pixel identifies a viewing ray, not a unique 3D location.

- Project camera-frame 3D points onto image pixels using camera intrinsics.

- Back-project a pixel using its known depth.

- Explain why camera-frame coordinates must be transformed into robot-base coordinates before commanding IK.

1. Pinhole camera model

For an ideal calibrated camera with positive depth Z and negligible lens distortion:

$u = fx * X/Z + cx$

$v = fy * Y/Z + cy$

(X,Y,Z) denotes a point in the camera coordinate frame; (u,v) denotes pixels; fx, fy are focal lengths in pixels; cx, cy are the principal point in pixels.

Given depth Z, the inverse relation is:

$X = (u - cx) * Z/fx$

$Y = (v - cy) * Z/fy$

Important: Pixel coordinates alone cannot provide unique depth. Points such as $(0.4,0.2,2.0)$ and $(0.8,0.4,4.0)$ can appear at the same pixel under the model.

2. Worked example

Camera intrinsics: $fx=fy=600$ px, $cx=320$ px, $cy=240$ px.

Camera-frame point: $(X,Y,Z)=(0.4,0.2,2.0)$ m.

Projection: $u = 600*(0.4/2)+320 = 440$ and $v = 600*(0.2/2)+240 = 300$.

With known depth $Z=2.0$ m, back-projection returns $X=0.4$ m, $Y=0.2$ m.

3. Python experiment

Save as lesson_17_camera_geometry.py:

```
import numpy as np

fx, fy = 600.0, 600.0
cx, cy = 320.0, 240.0
point_3d = np.array([0.4, 0.2, 2.0])
X, Y, Z = point_3d

u = fx * X / Z + cx
v = fy * Y / Z + cy

X_recovered = (u - cx) * Z / fx
Y_recovered = (v - cy) * Z / fy
recovered = np.array([X_recovered, Y_recovered, Z])
error = np.linalg.norm(recovered - point_3d)

print(f"Projected pixel: ({u:.1f}, {v:.1f})")
print("Recovered 3D point:", recovered)
```

```
print(f"Reconstruction error: {error:.3e} m")
assert np.allclose(recovered, point_3d, atol=1e-9)
print("Verification: PASSED")
```

Expected pixel: (440.0, 300.0); reconstructed point: (0.4, 0.2, 2.0) m; numerical verification: PASSED.

4. Robotics application

Image point + depth -> camera-frame 3D point -> camera-to-base rigid transform -> robot-base target -> inverse kinematics -> joint commands.

Lens distortion, depth noise, occlusion, calibration uncertainty, and extrinsic-calibration errors may affect real-world accuracy. A successful mathematical round trip is not evidence of real camera calibration quality.

5. Assessment record

- Python projection/back-projection: Passed (user-reported).

- First conceptual check on pixel ambiguity: Incorrect, then corrected through targeted reinforcement.

- Camera-to-robot coordinate transform: Correct.

- Final reinforcement: B — a pixel provides viewing direction but not a unique 3D distance/position. Correct.

Lesson 17 status: Completed.

Review questions

1. Why can two points at different depths project to the same pixel?

2. What extra information is needed to back-project a pixel into a unique 3D point?

3. Why cannot a camera-frame 3D point be sent directly to a base-frame IK solver?

Next lesson

Lesson 18: Camera-to-Robot Coordinate Transformations.